



Amlogic

ADLA NN SDK API Description

Revision: 1.2.0

Release Date: 2023-10-20

Copyright

© 2023 Amlogic. All rights reserved. No part of this document may be reproduced. Transmitted, transcribed, or translated into any language in any form or by any means with the written permission of Amlogic.

Trademarks

 , and other Amlogic icons are trademarks of Amlogic companies. All other trademarks and registered trademarks are property of their respective companies.

Disclaimer

Amlogic may make improvements and/or changes in this document or in the product described in this document at any time.

This product is not intended for use in medical, life saving, or life sustaining applications.

Circuit diagrams and other information relating to products of Amlogic are included as a means of illustrating typical applications. Consequently, complete information sufficient for production design is not necessarily given. Amlogic makes no representations or warranties with respect to the accuracy or completeness of the contents presented in this document.

Contact Information

- Website: www.amlogic.com
- Pre-sales consultation: contact@amlogic.com
- Technical support: support@amlogic.com

Reversion History

Issue 1.2.0 (2023-10-20)

Section	Change Description
2	Removed interfaces not supported by ADLA platform: aml_util_setProfile、aml_util_setPowerPolicy、aml_util_getHardwareStatus.
2.1.7	Improved the use of aml_util_getProfileInfo interface.
3.2	It is noted that the ADLA platform currently only supports the BINARY_RAW_DATA input mode after pre-processing.
3.5	Improved aml_util_enableProfile interface.

Issue 1.1.0 (2023-06-02)

Section	Change Description
1	Updated Introduction.

Issue 1.0.0 (2022-10-20)

This is the first release.

Contents

Reversion History	ii
1. Introduction	1
2. Introduce SDK API and data structure	3
2.1 Detailed description of SDK API	3
2.1.1 aml_module_create	3
2.1.2 aml_module_input_set	3
2.1.3 aml_module_output_get	4
2.1.4 aml_module_output_get_simple	4
2.1.5 aml_module_destroy	5
2.1.6 aml_util_enableProfile	5
2.1.7 aml_util_getProfileInfo	5
2.1.8 aml_util_disableProfile	6
2.1.9 aml_read_chip_info	6
2.1.10 aml_util_mallocBuffer	6
2.1.11 aml_util_freeBuffer	7
2.1.12 aml_util_swapExternalInputBuffer	7
2.1.13 aml_util_swapExternalOutputBuffer	8
2.1.14 aml_util_getTensorInfo	8
2.1.15 aml_util_freeTensorInfo	8
2.2 Detailed description of data structure of SDK	9
2.2.1 aml_config	9
2.2.2 assign_user_address_t	10
2.2.3 aml_output_config_t	10
2.2.4 aml_perf_mode_t	11
2.2.5 aml_output_order_t	11
2.2.6 aml_output_format_t	11
2.2.7 image_out_t	11
2.2.8 tensor_info	12
2.2.9 info	12
2.2.10 detBox	12
2.2.11 point_t	13
2.2.12 nn_output	13
2.2.13 outBuf_t	13

2.2.14 nn_buffer_params_t.....	13
2.2.15 nn_input.....	14
2.2.16 aml_profiling_data_t.....	15
2.2.17 aml_profile_config_t.....	15
2.2.18 aml_memory_config_t.....	15
2.2.19 aml_memory_data_t.....	16
3. Introduce how to use SDK API.....	17
3.1 Basic call process of SDK custom model.....	17
3.2 SDK demo call process description.....	17
3.2.1 Model initialization.....	17
3.2.2 set input.....	18
3.2.3 Get model results.....	19
3.3 Expansion function introduction.....	21
3.3.1 Introduction of SDK - DMA function.....	21
3.3.2 Introduction of SDK - usb camera function.....	22
3.3.3 Introduction of ffmpeg interface.....	23
3.4 reference code.....	23
3.5 Model performance debug interface.....	24

1. Introduction

This document introduces the Amlogic SDK API interface and related data structure to guide amlogic developers and amlogic customers.

Amlogic SDK path:

adla-toolkit-binary-1.x.x.x/bin/adla_plug_in/template/nnsdk

Amlogic SDK Demo path:

adla-toolkit-binary-1.x.x.x/bin/adla_plug_in/template/example

SDK API interface information is shown in table 1.1, SDK data structure information is shown in table 1.2.

Chapter one: The first chapter introduces the structure information and main contents of this document.

Chapter two: The SDK API and data structure are introduced, including the functions, parameters and return values of the SDK API. The data structure defined by the SDK is introduced in detail.

Chapter three: How to use the SDK API, this chapter introduces how to use the SDK API, and attaches a sample code for reference.

Table 1.1 SDK API interface table

SDK API	void* aml_module_create(aml_config* config);
	int aml_module_input_set(void* context, nn_input *pInput);
	void* aml_module_output_get(void* context, aml_output_config_t outconfig)
	void* aml_module_output_get_simple(void* context);
	int aml_module_destroy(void* context);
	int aml_util_getTensorInfo(void *context, const char* model_data, tensor_info** in_tinfo, tensor_info** out_tinfo);
	int aml_util_freeTensorInfo(tensor_info* tinfo);
	int aml_util_swapExternalInputBuffer(void *context, aml_memory_config_t* mem_config, aml_memory_data_t* mem_data);
	int aml_util_swapExternalOutputBuffer(void *context, aml_memory_config_t* mem_config, aml_memory_data_t* mem_data);
	int aml_util_mallocBuffer(void* context, aml_memory_config_t* mem_config, aml_memory_data_t* mem_data);
	int aml_util_freeBuffer(void* context, aml_memory_config_t* mem_config, aml_memory_data_t* mem_data);
	int aml_util_enableProfile(void *context, aml_profile_config_t* profile_data);
	int aml_util_getProfileInfo(void *context, aml_profile_config_t* profile_data);
	int aml_util_disableProfile(void *context, aml_profile_config_t* profile_data);
	int aml_read_chip_info(void);

Table 1.2 SDK API structure table

structure	function
aml_config	Output config information

assign_user_address_t	Set user defined memory address
aml_output_config_t	Set output model and data type information
aml_output_format_t	Set output data type
amlnn_input_mode_t	Set input mode
amlnn_model_type	Set framework type
amlnn_nbg_type	Set read nbg file type
amlnn_input_type	Set input data type
amlnn_query_cmd	Set query type
nn_buffer_format_e	Set output buffer type
nn_buffer_quantize_format_e	Set quantize type
aml_module_t	Set demo type
aml_profile_type_t	Set profile type
aml_policy_type_t	Set policy type
aml_flush_type_t	Set flush tensor type
assign_user_address_t	Set assign address type
image_out_t	Output image results
tensor_info	Output tensor information
nn_input	Set model input parameters
aml_output_config_t	Set model output config
aml_memory_config_t	Set the memory type parameter
aml_profile_config_t	Set profile type parameter

2. Introduce SDK API and data structure

2.1 Detailed description of SDK API

```

void* aml_module_create(aml_config* config);
int aml_module_input_set(void* context, nn_input *pInput);
void* aml_module_output_get(void* context, aml_output_config_t outconfig);
void* aml_module_output_get_simple(void* context);
int aml_module_destroy(void* context);
int aml_util_mallocBuffer(void* context, aml_memory_config_t* mem_config,
aml_memory_data_t* mem_data);
int aml_util_freeBuffer(void* context, aml_memory_config_t* mem_config, aml_memory_data_t*
mem_data);
int aml_util_swapExternalInputBuffer(void *context, aml_memory_config_t* mem_config,
aml_memory_data_t* mem_data);
int aml_util_swapExternalOutputBuffer(void *context, aml_memory_config_t* mem_config,
aml_memory_data_t* mem_data);
int aml_util_getTensorInfo(void *context, const char* model_data, tensor_info** in_tInfo,
tensor_info** out_tInfo);
int aml_util_freeTensorInfo(tensor_info* tinfo);
int aml_util_setProfile(aml_profile_type_t type, const char *savepath);
int aml_util_setPowerPolicy(aml_policy_type_t type);
int aml_util_getHardwareStatus(int* customID, int *powerStatus, int* version);
int aml_util_setAutoSuspend(int timeout);
int aml_util_enableProfile(void *context, aml_profile_config_t* profile_data);
int aml_util_getProfileInfo(void *context, aml_profile_config_t* profile_data);
int aml_util_disableProfile(void *context, aml_profile_config_t* profile_data);
int aml_read_chip_info(aml_platform_info_t* platform_info);

```

2.1.1 aml_module_create

API	void* aml_module_create(aml_config* config);
Function	Initialize the handle and load the generated nbg model
Parameter	Aml_config* config: path and type information of nbg model
Return value	Void*: handle context object pointer

Code example:

```

void *context = NULL;
aml_config config;
memset(&config, 0, sizeof(aml_config));

config.path = nb_name;
config.nbgType = NN_ADLA_FILE;
config.typeSize = sizeof(aml_config);
config.modelType = ADLA_LOADABLE;

context = aml_module_create(&config);

```

2.1.2 aml_module_input_set

API	Int aml_module_input_set(void* context, nn_input *pInput);
Function	Set the input data according to the nbg model
Parameter	void* context: handle context pointer

	nn_input *pInput: input data. Currently, only one input can be set at a time. If it is a multiple input model, you need to call amlnn_inputs_set interface several times, which will be corrected later.
Return value	int: success returns zero, and failure returns nonzero.

Code example:

```
nn_input inData;
unsigned char *rawdata;

rawdata = get_jpeg_rawData(jpeg_path, 224, 224);
inData.input_index = 0; //this value is index of input, begin from 0
inData.size = 224*224*3;
inData.input = rawdata;
inData.input_type = RGB24_RAW_DATA;
ret = aml_module_input_set(context, &inData);
```

2.1.3 aml_module_output_get

API	void* aml_module_output_get(void* context, aml_output_config_t outconfig)
Function	obtain the result of one inference.
Parameter	void* context: handle context pointer.
Return value	aml_output_config_t outconfig: model type and output type.
	void*: Inference result structure pointer.

Code example:

```
nn_output *outdata = NULL;
modelType = CUSTOM_NETWORK;
outconfig.format = AML_OUTDATA_FLOAT32;
outdata = (nn_output*)aml_module_output_get(context, outconfig);
process_top5((float*)outdata->out[0].buf, outdata->out[0].size/sizeof(float), NULL);
```

2.1.4 aml_module_output_get_simple

API	void* aml_module_output_get_simple(void* context);
Function	obtain the result of one inference, output tensor type is float32
Parameter	void* context: handle context pointer.
Return value	void*: Inference result structure pointer.

Code example:

```
nn_output *outdata = NULL;
outdata = (nn_output *)aml_module_output_get_simple(qcontext);
process_top5((float*)outdata->out[0].buf, outdata->out[0].size/sizeof(float), NULL);
```

2.1.5 aml_module_destroy

API	int aml_module_destroy(void* context);
Function	unload the nbg model, and destroy the context and related resources.
Parameter	void* context: handle context pointer.
Return value	int: Success returns zero, and failure returns nonzero.

Code example:

```
int ret = aml_module_destroy(context);
```

2.1.6 aml_util_enableProfile

API	int aml_util_enableProfile(void *context, aml_profile_config_t* profile_data);
Function	profile performance debugging function enabled
Parameter	void *context: handle context pointer aml_profile_config_t* profile_data: Performance parameter information structure
Return value	int: Success returns zero, and failure returns nonzero.

Code example:

```
context = aml_module_create(&config);
aml_profile_config_t profile_data;
aml_util_enableProfile(context, &profile_data);
```

2.1.7 aml_util_getProfileInfo

API	int aml_util_getProfileInfo(void *context, aml_profile_config_t* profile_data);
Function	Get profile performance parameter information
Parameter	void *context: handle context pointer aml_profile_config_t* profile_data: Performance parameter information structure
Return value	int: Success returns zero, and failure returns nonzero.

Code example:

```
context = aml_module_create(&config);
aml_profile_config_t profile_data;
aml_util_getProfileInfo(context, &profile_data);
printf("\nProfile data:\n");
printf("inference_time: %d ms\n", profile_data.profiling_data.inference_time_us);
printf("memory_usage: %d Mbytes\n", profile_data.profiling_data.memory_usage_bytes);
printf("dram_read_bytes: %d Mbytes\n", profile_data.profiling_data.dram_read_bytes);
printf("dram_write_bytes: %d Mbytes\n", profile_data.profiling_data.dram_write_bytes);

printf("axi_freq_cur: %d \n", profile_data.profiling_data.ext.axi_freq_cur);
printf("core_freq_cur: %d \n", profile_data.profiling_data.ext.core_freq_cur);
printf("mem_allocated_base: %d \n", profile_data.profiling_data.ext.axi_freq_cur);
printf("mem_allocated_umd: %d \n", profile_data.profiling_data.ext.mem_allocated_umd);
```

```
printf("mem_pool_size: %d \n", profile_data.profiling_data.ext.mem_pool_size);
printf("mem_pool_used: %d \n", profile_data.profiling_data.ext.mem_pool_used);
printf("us_elapsed_in_hw_op: %d \n", profile_data.profiling_data.ext.us_elapsed_in_hw_op);
printf("us_elapsed_in_sw_op: %d \n", profile_data.profiling_data.ext.us_elapsed_in_sw_op);
aml_util_disableProfile(context, &profile_data);
```

2.1.8 aml_util_disableProfile

API	int aml_util_disableProfile(void *context, aml_profile_config_t* profile_data);
Function	profile performance debugging is turned off
Parameter	void *context: handle context pointer
	aml_profile_config_t* profile_data: Performance parameter information structure
Return value	int: Success returns zero, and failure returns nonzero.

Code example:

```
context = aml_module_create(&config);
aml_profile_config_t profile_data;
aml_util_disableProfile(context, &profile_data);
```

2.1.9 aml_read_chip_info

API	int aml_read_chip_info(aml_platform_info_t* platform_info);
Function	Read platform NPU hardware information
Return value	int: Success returns zero, and failure returns nonzero.

Code example:

```
aml_platform_info_t platform_info;
aml_read_chip_info(&platform_info);
Printf("sdk_version:%s\n", profile_data.sdk_version);
```

2.1.10 aml_util_mallocBuffer

API	int aml_util_mallocBuffer(void *context, aml_memory_config_t* mem_config, aml_memory_data_t* mem_data);
Function	Allocate memory space for inbuf or outbuf to use input/output DMA function
Parameter	void *context: handle context pointer
	aml_memory_config_t* mem_config: Memory allocation configuration information
	aml_memory_data_t* mem_data: Driver allocated memory information
Return value	int: Success returns zero, and failure returns nonzero.

Code example:

```
aml_memory_config_t mem_config;
```

```

aml_memory_data_t mem_data;
mem_config.index = 0;
mem_config.mem_size = inData.size;
aml_util_mallocBuffer(context, &mem_config, &mem_data);

rawdata = get_jpeg_rawData(input_path, 224, 224);
memcpy(mem_data.viraddr, rawdata, inData.size);
ret = aml_module_input_set(qcontext, &inData);

```

2.1.11 aml_util_freeBuffer

API	int aml_util_freeBuffer(void *context, aml_memory_config_t* mem_config, aml_memory_data_t* mem_data);
Function	Free the inbuf dma or outbuf dma memory space allocated using aml_util_mallocBuffer
Parameter	void *context: handle context pointer
	aml_memory_config_t* mem_config,: Memory release configuration information
	aml_memory_data_t* mem_data: Memory information to be released
Return value	int: Success returns zero, and failure returns nonzero.

Code example:

```

aml_memory_config_t mem_config;
aml_memory_data_t mem_data;
aml_util_mallocBuffer(context, &mem_config, &mem_data);

```

2.1.12 aml_util_swapExternalInputBuffer

API	int aml_util_swapExternalInputBuffer(void *context, aml_memory_config_t* mem_config, aml_memory_data_t* mem_data);
Function	Modify the start pointer of the DMA buffer corresponding to the input physical address to realize fast switching of input
Parameter	void *context: handle context pointer
	aml_memory_config_t* mem_config,: Memory switch configuration information
	aml_memory_data_t* mem_data: Memory information to be switched
Return value	int: Success returns zero, and failure returns nonzero.

Code example:

```

aml_memory_config_t mem_config;
aml_memory_data_t mem_data;

mem_config.index = 0;
mem_config.mem_size = inData.size;
rawdata = get_jpeg_rawData(input_path, 224, 224);
memcpy(mem_data.viraddr, rawdata, inData.size);

```

```
aml_util_swapExternalIoBuffer(context, mem_config, mem_data);
```

2.1.13 aml_util_swapExternalOutputBuffer

API	int aml_util_swapExternalOutputBuffer(void *context, aml_memory_config_t* mem_config, aml_memory_data_t* mem_data);
Function	Modify the start pointer of the DMA buffer corresponding to the output physical address to achieve fast switching of output
Parameter	void *context: handle context pointer
	aml_memory_config_t* mem_config,: Memory switch configuration information
	aml_memory_data_t* mem_data: Memory information to be switched
Return value	int: Success returns zero, and failure returns nonzero.

2.1.14 aml_util_getTensorInfo

API	int aml_util_getTensorInfo(void *context, const char* model_data, tensor_info** in_tInfo, tensor_info** out_tInfo);
Function	Get the input and output information of the adla file
Parameter	void *context: handle context pointer
	const char* model_data,: adla file data information
	tensor_info** in_tInfo: pointer to input tensor information in adla
	tensor_info** out_tInfo: pointer to output tensor information in adla
Return value	int: Success returns zero, and failure returns nonzero.

Code example:

```
tensor_info* in_tInfo = NULL;
tensor_info* out_tInfo = NULL;
aml_util_getTensorInfo(context, config.pdata, &in_tInfo, &out_tInfo);
{
    printf("the input buffer info is get success\n");
    printf("the input tensor number is %d\n", inptr->num);
    printf("the input[0] data_format:%d\n", inptr->info[0].data_format);
    printf("the input[0] quantization_format:%d\n", inptr->info[0].quantization_format);
    printf("the input[0] sizes_of_dim:%d,%d,%d,%d\n", inptr->info[0].sizes_of_dim[0],
    inptr->info[0].sizes_of_dim[1], inptr->info[0].sizes_of_dim[2], inptr->info[0].sizes_of_dim[3]);
}
aml_util_freeTensorInfo(in_tInfo);
aml_util_freeTensorInfo(out_tInfo);
```

2.1.15 aml_util_freeTensorInfo

API	int aml_util_freeTensorInfo(tensor_info* tinfo);
Function	Free the allocated inptr or outptr memory space
Parameter	tensor_info*: inptr or outptr data obtained by getTensorInfo
Return value	int: Success returns zero, and failure returns nonzero.

Code example:

```

tensor_info* in_tInfo = NULL;
tensor_info* out_tInfo = NULL;
aml_util_getTensorInfo(context, config.pdata,& in_tInfo, &out_tInfo);
{
    printf("the input buffer info is get success\n");
    printf("the input tensor number is %d\n",inptr->num);
    printf("the input[0] data_format:%d\n",inptr->info[0].data_format);
    printf("the input[0] quantization_format:%d\n",inptr->info[0].quantization_format);
    printf("the input[0] sizes_of_dim:%d,%d,%d,%d\n",inptr->info[0].sizes_of_dim[0],
    inptr->info[0].sizes_of_dim[1],inptr->info[0].sizes_of_dim[2],inptr->info[0].sizes_o
f_dim[3]);
}
aml_util_freeTensorInfo(in_tInfo);
aml_util_freeTensorInfo(out_tInfo);

```

2.2 Detailed description of data structure of SDK

2.2.1 aml_config

Member variable	Data type	Meaning
typeSize	int	Number of structure member variables
path	const char *	load nbg from file pointer
pdata	const char *	load nbg from memory pointer
length	int	nbg size
modelType	amlInn_model_type	Nbg model type
nbgType	amlInn_nbg_type	load nbg method
inOut	assign_user_address_t	assign user address
forward_ctrl	aml_forward_ctrl_t	Model inference control options
compiler_args	aml_compiler_args_t	Model build control options

```

typedef struct __aml_nn_config
{
    int typeSize;
    int length;
    const char *path;
    const char *pdata;
    amlInn_model_type modelType;
    amlInn_nbg_type nbgType;
    assign_user_address_t inOut;
    aml_forward_ctrl_t forward_ctrl;
    aml_compiler_args_t compiler_args;
} aml_config;

typedef enum _amlInn_nbg_type_ {
    NN_NBG_FILE = 0,
    NN_NBG_MEMORY = 1,
    NN_RUNTIME_FILE = 2,
    NN_RUNTIME_MEMORY = 3,
    NN_ADLA_FILE = 4,
    NN_ADLA_MEMORY = 5
} amlInn_nbg_type;

typedef struct __aml_forward_ctrl_t
{
    aml_ensure_id          enCoreId;      /* device target which running the seg*/
    int64_t                invoke_id;

```

```

    int32_t          timeout_ms;
} aml_forward_ctrl_t;

typedef struct __aml_compiler_args_t
{
    int32_t batch_multiplier;
    int32_t compiler_only;
} aml_compiler_args_t;

```

amlnn_model_type

Member variable	Model type
CAFFE	Caffe model
TENSORFLOW	Tensorflow model
TENSORFLOWLITE	TensorflowLite model
DARKNET	Darknet model
ONNX	Onnx model
KERAS	Keras model
PYTORCH	Pytorch model
ADLA_LOADABLE	Adla model

2.2.2 assign_user_address_t

Member variable	Data type	Meaning
io_type	aml_io_format_t	dma io type
inAddr	unsigned char*	input address
outAddr	unsigned char*	output address

```

typedef struct __assign_address
{
    unsigned char* inAddr[ADDRESS_MAX_NUM];
    unsigned char* outAddr[ADDRESS_MAX_NUM];
} assign_user_address_t;

typedef enum {
    AML_IO_VIRTUAL = 0,
    AML_IO_PHYS = 1,
} aml_io_format_t;

```

2.2.3 aml_output_config_t

Member variable	Data type	Meaning
typeSize	int	Number of structure member variables
mdType	aml_module_t	Network model
perfMode	aml_perf_mode_t	Model inference type
format	aml_output_format_t	Output data type
order	aml_output_order_t	Output data arrangement, NCHW/NHWC

```

typedef struct __amlnn_module_out_data_t
{
    int typeSize;
}

```

```

aml_module_t mdType;
aml_perf_mode_t perfMode;
aml_output_format_t format;
aml_output_order_t order;
} aml_output_config_t;

```

2.2.4 aml_perf_mode_t

Member variable	Meaning
AML_NO_PERF	Performance test mode off
AML_PERF_INFERENCE	Inference step
AML_PERF_OUTPUT	inverse quantization step

```

typedef enum {
    AML_NO_PERF          = 0,
    AML_PERF_INFERENCE   = 1,
    AML_PERF_OUTPUT      = 2
} aml_perf_mode_t;

```

2.2.5 aml_output_order_t

Member variable	Meaning
AML_OUTPUT_ORDER_DEFAULT	adla is output in the order of NHWC, galcore is output in the order of NCHW
AML_OUTPUT_ORDER_NHWC	Output in the order of NHWC
AML_OUTPUT_ORDER_NCHW	Output in the order of NCHW

```

typedef enum {
    AML_OUTPUT_ORDER_DEFAULT = 0,    //output format:  nhwc for adla, nchw for galcore
    AML_OUTPUT_ORDER_NHWC    = 1,    //output format: nhwc
    AML_OUTPUT_ORDER_NCHW    = 2,    //output format: nchw
} aml_output_order_t;

```

2.2.6 aml_output_format_t

Member variable	Meaning
AML_OUTDATA_FLOAT32	Output data of type float32
AML_OUTDATA_RAW	Output raw data
AML_OUTDATA_DMA	Output dma data

```

typedef enum {
    AML_OUTDATA_FLOAT32 = 0,
    AML_OUTDATA_RAW     = 1,
    AML_OUTDATA_DMA     = 2
} aml_output_format_t;

```

2.2.7 image_out_t

Member variable	Data type	Meaning
height	int	Output image height value
width	int	Output image width value
channel	int	Number of output image channels
data	unsigned char *	Output image buffer information


```
typedef struct __nn_image_out
{
    int height;
    int width;
    int channel;
    unsigned char *data; //this buffer is returned by aml_module_output_get
}image_out_t;
```

2.2.8 tensor_info

Member variable	Data type	Meaning
valid	unsigned int	Get tensor information flag
num	unsigned int	Tensor number
info	info *	tensor information pointer

```
typedef struct {
    unsigned int valid;
    unsigned int num;
    info *info;
} tensor_info;
```

2.2.9 info

Member variable	Data type	Meaning
dim_count	unsigned int	Number of dimensions
sizes_of_dim	unsigned int[]	Dimension value
data_format	unsigned int	buffer format type
data_type	unsigned int	Data type
quantization_format	unsigned int	Quantitative type
fixed_point_pos	int	int8/int16 Quantified parameter values
TF_scale	float	Scale value
TF_zeropoint	int	Zeropoint value
name	char[]	name

```
typedef struct {
    unsigned int dim_count; /*dim count*/
    unsigned int sizes_of_dim[4]; /*dim value,just support 4-d dim*/
    unsigned int data_format; /*see as nn_buffer_format_e*/
    unsigned int data_type; /*not use*/
    unsigned int quantization_format; /*see as nn_buffer_quantize_format_e*/
    int fixed_point_pos; /*for int8/int16 UANTIZE_DYNAMIC_FIXED_POINT*/
    float TF_scale; /*as tf define,scale*/
    int TF_zeropoint; /*as tf define,zeropoint*/
    char name[MAX_NAME_LEGTH]; /*not use,will used in future*/
} info
```

2.2.10 detBox

Member variable	Data type	Meaning
x	float	Output the x-axis coordinate value of the object box
y	float	Output the Y-axis coordinate value of the object box
w	float	Output the width information of the object box

h	float	Output the height information of the object box
score	float	Output the score information of the object
objectClass	float	Output the category information of the object

2.2.11 point_t

Member variable	Data type	Meaning
x	float	Output the X-axis coordinate value of the object point
y	float	Output the Y-axis coordinate value of the object point

2.2.12 nn_output

Member variable	Data type	Meaning
typeSize	int	Number of structure member variables
num	unsigned int	Number of output tensors
out	outBuf_t[]	Output tensor structure array

```
typedef struct __nnout
{
    int typeSize;
    unsigned int num; /*=====output tensor number=====*/
    outBuf_t out[OUTPUT_MAX_NUM];
}nn_output;
```

2.2.13 outBuf_t

Member variable	Data type	Meaning
size	unsigned int	Number of output tensors
buf	unsigned char *	Output tensor data address
param	nn_buffer_params_t *	Output properties of tensor
name	char	Output tensor name

```
typedef struct out_buf
{
    unsigned int size;
    unsigned char *buf;
    nn_buffer_params_t *param;
    char name[MAX_NAME_LENGTH]; //output tensor name
    aml_output_format_t out_format;
}outBuf_t;
```

2.2.14 nn_buffer_params_t

Member variable	Data type	Meaning
num_of_dims	unsigned int	Number of tensor dimensions
sizes	unsigned int[]	Values of tensor dimensions
data_format	nn_buffer_format_e	There are several data types of tensor: NN_BUFFER_FORMAT_FP32 = 0, NN_BUFFER_FORMAT_FP16 NN_BUFFER_FORMAT_UINT8 NN_BUFFER_FORMAT_INT8

		NN_BUFFER_FORMAT_UINT16 NN_BUFFER_FORMAT_INT16
quant_format	nn_buffer_quantize_format_e	The quantization format has the following values: NN_BUFFER_QUANTIZE_NONE=0, NN_BUFFER_QUANTIZE_DYNAMIC_FIXED_POINT (dynamic fixed point quantization) NN_BUFFER_QUANTIZE_TF_ASYMM (Asymmetric quantization)
dfp	struct	DYNAMIC_FIXED_POINT quantitative type parameter: fl = dfp.fixed_point_pos
affine	struct	TF_ASYMM quantitative type parameter scale=affine.scale zeroPoint=affine.zeroPoint

```

typedef struct _nn_buffer_create_params_t
{
    unsigned int    num_of_dims;
    unsigned int    sizes[4];
    nn_buffer_format_e    data_format;
    nn_buffer_quantize_format_e quant_format;
    union {
        struct {
            unsigned char fixed_point_pos;
        } dfp;
        struct {
            float    scale;
            unsigned int zeroPoint;
        } affine;
    }
    quant_data;
} nn_buffer_params_t;

```

2.2.15 nn_input

Member variable	Data type	Meaning
typeSize	int	Number of structure member variables
input_index	int	The input data corresponds to the subscript of tensor, and the single input is 0
size	int	Input data size
input	unsigned char*	Input data address
input_type	aml_nn_input_type	Input data types include the following types: RGB24_RAW_DATA = 0, TENSOR_RAW_DATA, QTENSOR_RAW_DATA, BINARY_RAW_DATA INPUT_DMA_DATA
info	input_info	Input buffer information

```

typedef struct {
    int valid;
    float mean[INPUT_CNANNEL];
    float scale;
    aml_input_format_t input_format;
    int int16_type ; //when int16 quantize model mean value is not 128,128,128,set 1

```

```
int preprocess_debug; // save npu input data
}input_info;
typedef enum {
    AML_INPUT_DEFAULT      = 0,    //channle format: caffe 2 1 0 ,others 0 1 2
    AML_INPUT_MODEL_NHWC   = 1,    //channle format: 0 1 2
    AML_INPUT_MODEL_NCHW   = 2,    //channle format: 2 1 0
} aml_input_format_t;
```

2.2.16 aml_profiling_data_t

Member variable	Data type	Meaning
inference_time_us	int	Inference time
memory_usage_bytes	int	Overall memory usage
memory_read_bytes	int	Read memory usage
memory_write_bytes	int	Write memory usage
ext	aml_profiling_ext_data_t	Additional performance information

```
typedef struct aml_profiling_ext_data
{
    unsigned int axi_freq_cur;
    unsigned int core_freq_cur;
    unsigned int mem_allocated_base;
    unsigned int mem_allocated_umd;
    int mem_pool_size;
    unsigned int mem_pool_used;
    int us_elapsed_in_hw_op;
    int us_elapsed_in_sw_op;
} aml_profiling_ext_data_t;
```

2.2.17 aml_profile_config_t

Member variable	Data type	Meaning
suspend_time	unsigned int	Context memory usage information
freq	float	Context model layer information
profile_type	aml_profile_type_t	Type of performance information
policy_type	aml_policy_type_t	Power Policy Information
profiling_data	aml_profiling_data_t	performance information
frequency_data	aml_work_frequency_t	frequency information

```
typedef struct aml_work_frequency
{
    float *curr_freq;
    float *min_freq;
} aml_work_frequency_t;
```

2.2.18 aml_memory_config_t

Member variable	Data type	Meaning
typeSize	int	structure size
mmu_type	float	MMU status
cache_type	aml_cache_type_t	Cache mode
memory_type	aml_memory_type_t	memory status
profiling_data	aml_profiling_data_t	performance information
frequency_data	aml_work_frequency_t	frequency information

```
typedef struct __aml_memory_config_t
```

```

{
    uint32_t      typeSize;
    uint32_t      mem_size;
    uint32_t      index;
    aml_mmu_type_t mmu_type;
    aml_cache_type_t cache_type;
    aml_memory_type_t memory_type;
    aml_mem_direction_t direction;
} aml_memory_config_t;

```

2.2.19 aml_memory_data_t

Member variable	Data type	Meaning
typeSize	int	structure size
memory	void*	Kernel layer virtual address
viraddr	void*	user layer virtual address
phyaddr	uint64_t	Kernel layer physical address

```

typedef struct __aml_memory_data_t
{
    uint32_t      typeSize;
    void          *memory;
    void          *viraddr;
    uint64_t      phyaddr;
} aml_memory_data_t;

```

3. Introduce how to use SDK API

Currently, the SDK package uses a specific network ID. CUSTOM_NETWORK represents the new network, so the output data will retain the output of the original network, and no post-processing operation will be done for this type. In multiple network application, customers can manage multiple network instances according to the context value returned by network aml_module_createa.

Please refer to GitHub for the demo demo of each function usage mode:
https://github.com/Amlogic-NN/AML_NN_SDK/tree/master/Linux/Demo

3.1 Basic call process of SDK custom model

1. Initialization model: `void* context = aml_module_create(&config);`
2. Prepare input data and set input data:
`inData.input_index = 0;`
`inData.size = 224*224*3;`
`inData.input = rawdata;`
`inData.input_type = INPUT_DMA_DATA;`
`int ret = aml_module_input_set(context,&inData);`
3. Carry out forward inference and obtain the inference results:
`outdata = (nn_output *)aml_module_output_get_simple(context);`
4. Multiple inference: repeat steps 6 and 7 several times
5. Unload model: `int ret = aml_module_destroy(context);`

3.2 SDK demo call process description

3.2.1 Model initialization

aml_config config;

1. model adla has two load methods

Mode one: load adla from memory

```
fp = fopen(argv[1],"rb");
fseek(fp,0,SEEK_END);
size = (int)ftell(fp);
rewind(fp);
config.pdata = (char *)calloc(1,size);
fread((void*)config.pdata,1,size,fp);
config.nbgType = NN_ADLA_MEMORY;
config.length = size;
fclose(fp);
```

Mode two: load adla from file

```
config.path= (const char *)argv[1];
config.nbgType = NN_ADLA_FILE;
```

2. model runtime has two load methods

Mode one: load runtime from memory

```
fp = fopen(argv[1], "rb");
fseek(fp, 0, SEEK_END);
size = (int)ftell(fp);
rewind(fp);
config.pdata = (char *)calloc(1, size);
fread((void*)config.pdata, 1, size, fp);
config.nbgType = NN_RUNTIME_MEMORY;
config.length = size;
fclose(fp);
```

Mode two: load runtime from file

```
config.path = (const char *)argv[1];
config.nbgType = NN_RUNTIME_FILE;
config.modelType = ADLA_LOADABLE;
void *context = aml_module_create(&config);
```

Notes:

- The input parameter of file mode is the path of adla and runtime files. Among them, config.path is a pointer to the specific information of adla. When the specified config.nbgType is NN_RUNTIME_FILE and NN_ADLA_FILE, the assignments correspond to the paths of the runtime and adla files respectively.
- When the model file content is already saved in memory, you can choose to load from memory. Among them, config.pdata is a pointer to the specific information of runtime and adla. When config.nbgType is specified as NN_***_MEMORY, its assignment represents the memory first address of the model file. Different from NN_ADLA_FILE, this method needs to additionally set config.size to the actual length of the memory of the model file.
- The way to use memory to load model files requires memory release, free((void*)config.pdata).

3.2.2 set input

```
nn_input inData;
unsigned char *rawdata = read_file(input_path, &input_size); //Read input data
inData.input_index = 0; //Input data index value to support multiple data input
inData.size = 416*416*3; //Set input size, W*H*C
inData.input = rawdata; //Set input buffer
inData.input_type = BINARY_RAW_DATA; //Set input data format
```

Notes:

- read_file: Read the input file after preprocessing.
- inData.size: Integrated model input data, size determined.
- input_index: Input the data index value. Single input can be set to 0. In case of multiple inputs, the interface needs to be called multiple times, and the index value is increased by 1 in turn.
- inData.input_type: Input data types include the following types:

Note: The ADLA platform currently only supports the BINARY_RAW_DATA input mode after pre-processing.

Input data type	meaning	Processing method
RGB24_RAW_DATA	Input RGB data, (0,255)	Reorder,mean,scale
TENSOR_RAW_DATA	Input tensor float data, Processing method is float32	Memcpy(&fval,rawdata,sizeof(float)); convertFloatToDtype(fval,tensordata)

QTENSOR_RAW_DATA	Input tensor float data, Processing method is uint8	Memcpy(&fval,rawdata,sizeof(float)); Tensordata = (uint8)fval
BINARY_RAW_DATA	Input rawdata	Memcpy(tensordata_all,rawdata_all,all_size);

In addition, type *INPUT_DMA_DATA* is supported, if using input dma function, *inData*. *input_type* needs to be set to this type.

Input_type type description:

- *RGB24_RAW_DATA*: Input RGB format data. After data input, the system will do reorder, mean, scale and other related operations. The values of these parameters come from the information set when acuity generates nbg.
- *TENSOR_RAW_DATA*: Float input of raw tensor data. After getting the data buffer, the system will perform the following operations:

Float fval;

Memcpy(&fval,rawdata,sizeof(float));

convertFloatToDtype(fval,tensordata);

The above completes a float data processing. Where *rawdata* is the input raw data, *tensordata* is the final input data into the model.

- *QTENSOR_RAW_DATA*: Be similar to *TENSOR_RAW_DATA*. The input data is of float type, and the processing method is uint8:

Float fval;

Memcpy(&fval,rawdata,sizeof(float));

Tensordata = (uint8)fval;

The above completes a float data processing. Where *rawdata* is the input raw data, *tensordata* is the final input data into the model.

- *BINARY_RAW_DATA*: The raw data is input without any processing and conversion. The process is approximately as follows:

Memcpy(tensordata_all,rawdata_all,all_size);

Copy all the input data intact into the input buffer of the model.

- *INPUT_DMA_DATA*: Input obtained using DMA function.

data = __get_rgb_data(tensor, pmeta, input, format_type);

memcpy(input_dma,data,sz*stride)

3.2.3 Get model results

Use the output structure of *nn_output* to receive the result returned by the *aml_module_output_get* interface.

```
nn_output *outdata = NULL;
outconfig.typeSize = sizeof(aml_output_config_t);
modelType = CUSTOM_NETWORK; //Set output model type
outconfig.format = AML_OUTDATA_FLOAT32; //Set output data type
outdata = (nn_output*)aml_module_output_get(context,outconfig);
if(outdata->out[0].param->data_format == NN_BUFFER_FORMAT_FP32)
{
process_top5((float*)outdata->out[0].buf,outdata->out[0].size/sizeof(float),NULL);
}
```


Outdata contains complete output information, including output dim, output format, output content and so on. The output content is float32 format by default, which can be implemented after relevant network post-processing. Such as mobilenet:

```
process_top5((float*)pout->out[0].buf,pout->out[0].size/sizeof(float));
```

Notes:

- *Outconfig.format can choose AML_OUTDATA_FLOAT32, AML_OUTDATA_RAW, AML_OUTDATA_DMA according to actual needs.*
- *AML_OUTDATA_FLOAT32 indicates that the output data type is f32, and the type conversion has been completed inside the SDK; AML_OUTDATA_RAW indicates the output of the original type of data; AML_OUTDATA_DMA indicates the output dma output buffer data type, which needs to be used with DMA.*
- *In order to facilitate users to use custom models, a simple parameter interface is provided for forward reasoning and obtaining reasoning results.*

```
void* aml_module_output_get_simple(void* context);
```

This interface only needs to pass in the context, and the default output data type is float32. usage:

```
nn_output * outdata = (nn_output *)aml_module_output_get_simple(context);
```

Users can flexibly choose the reasoning interface according to their needs.

- *If the user needs to obtain the forward reasoning time of the model, the performance test mode can be enabled through outconfig.perfMode to obtain the forward reasoning and dequantization time respectively. outconfig.perfMode is not set by default, and the test mode is closed by default.*

```
outconfig.typeSize = sizeof(aml_output_config_t);
modelType = CUSTOM_NETWORK;
outconfig.format = AML_OUTDATA_FLOAT32;
tmsStart = get_perf_count();
outconfig.perfMode = AML_PERF_INFERENCE;
for (i=0;i<100;i++)
    outdata = (nn_output*)aml_module_output_get(qcontext,outconfig);
tmsEnd = get_perf_count();
msVal = (tmsEnd - tmsStart)/1000000;
usVal = (tmsEnd - tmsStart)/1000;
printf("AML_PERF_INFERENCE: %ldms or %ldus\n", msVal, usVal);

tmsStart = get_perf_count();
outconfig.perfMode = AML_PERF_OUTPUT;
outdata = (nn_output*)aml_module_output_get(qcontext,outconfig);
tmsEnd = get_perf_count();
msVal = (tmsEnd - tmsStart)/1000000;
usVal = (tmsEnd - tmsStart)/1000;
printf("AML_PERF_OUTPUT: %ldms or %ldus\n", msVal, usVal);
```

3.3 Expansion function introduction

3.3.1 Introduction of SDK - DMA function

3.3.1.1 input dma set

Virtual Address

```
aml_memory_config_t mem_config;
aml_memory_data_t mem_data;
input_size = 224*224*3;
mem_config.mem_size = input_size;
mem_config.index = 0;
mem_config.memory_type = AML_VIRTUAL_ADDR;
aml_util_mallocBuffer(context, &mem_config, &mem_data);
rawdata = get_jpeg_rawData(input_path, 224, 224);
memcpy(mem_data.viraddr, rawdata, inData.size);
aml_util_swapExternalInputBuffer(context, mem_config, mem_data)
```

Note:

- *aml_util_mallocBuffer*, The size of the corresponding image needs to be input when allocating memory.

Physical Address

```
aml_memory_config_t mem_config;
aml_memory_data_t mem_data;
input_size = 224*224*3;
mem_config.mem_size = input_size;
mem_config.index = 0;
mem_config.memory_type = AML_PHYS_ADDR;;
aml_util_mallocBuffer(context, &mem_config, &mem_data);
rawdata = read_file(input_path, &input_size);
memcpy(mem_data.viraddr, rawdata, inData.size);
aml_util_swapExternalInputBuffer(context, mem_config, mem_data);
```

Note:

- *ainput_index* is *inputId*, starting from 0.
- This operation can be prepared during the execution of the first frame, forming an effect similar to ping-pong buffer, so as to save the input input time.

3.3.1.2 output dma set

Virtual Address

```
aml_memory_config_t mem_config;
aml_memory_data_t mem_data;
mem_config.mem_size = output_size;
mem_config.index = 0;
mem_config.memory_type = AML_VIRTUAL_ADDR;;
aml_util_mallocBuffer(context, &mem_config, &mem_data);
aml_util_swapExternalOutputBuffer(context, mem_config, mem_data);
```

Physical Address

```
output_size = 2*1000;
aml_memory_config_t mem_config;
aml_memory_data_t mem_data;
```

```
mem_config.mem_size = output_size;
mem_config.index = 0;
mem_config.memory_type = AML_PHYS_ADDR;;
aml_util_mallocBuffer(context, &mem_config, &mem_data);
aml_util_swapExternalOutputBuffer(context, mem_config, mem_data);
```

Notes:

- *aml_util_mallocBuffer needs to set the output buffer size. For example, the classification result of the classification model is 1000 types, and each type is of float16 type, so the required length of outbuf is 1000*2.*

3.3.2 Introduction of SDK - usb camera function

USB camera can be displayed on framebuffer in real time by calling camera_thread_func and net_thread_func two threads.

```
context = init_network(argc,argv);
pthread_mutex_init(&mutex_data,NULL);
init_fb();
if (0 != pthread_create(&tid[0],NULL,camera_thread_func,NULL))
{
    fprintf(stderr, "Couldn't create thread func\n");
    return -1;
}
thread_args = (void*)argv[3];
if (0 != pthread_create(&tid[1],NULL,net_thread_func,thread_args))
{
    fprintf(stderr, "Couldn't create thread func\n");
    return -1;
}
while(1)
{
    for (i=0;i<2;i++)
        pthread_join(tid[i], NULL);
}
```

Among them, camera_thread_func integrates the calling mode of usb camera. net_thread_func copies the video stream information read from the camera, and uses it as the input information of the model operation after resize.

```
dup_rgbbuf = (unsigned char*)malloc(display_width * display_high * 3);
input_data = (unsigned char*)malloc(input_width * input_high * 3);
while(1)
{
    if(rgbbuf != NULL)
    {
        pthread_mutex_lock(&mutex_data);
        memcpy(dup_rgbbuf,rgbbuf,display_width*display_high*3);
        pthread_mutex_unlock(&mutex_data);
    }
    resize_input_data(dup_rgbbuf,input_data);
    ret = run_network(netType,context,input_data,AML_IN_CAMERA,dup_rgbbuf);
}
```

Note:

- *The current resolution is set to 640 * 480 by default, and supports 1920 * 1080, 1280 * 720 and 640 * 480 resolutions.*

3.3.3 Introduction of ffmpeg interface

SDK supports input video data, and uses ffmpeg to decode the video for network input. The image obtained by parsing the video and resize is fed into the model as rawdata input data.

```
sprintf(cmd,"ffmpeg -i %s -qscale:v 2 -r 24 ",jpath);
ptr=strcat(cmd,"tmp/image%5d.bmp");
system(ptr);
sprintf(img_name,"tmp/image%05d.bmp",index);
while (1)
{
    sprintf(img_name,"tmp/image%05d.bmp",index);
    if((access(img_name,F_OK)) < 0)
    {
        break;
    }
    img=imread(img_name,199);
    resize(img,dst,dst.size());
    ret = run_network(netType,context,dst.data,AML_IN_VIDEO,(unsigned char*)img_name);
    index++;
}
system("ffmpeg -f image2 -i tmp/image%5d.bmp -b:v 5626k videoout.mp4");
```

3.4 reference code

```
int main(int argc,char **argv)
{
    void *context;
    const char *jpeg_path = NULL;
    unsigned char *rawdata;
    aml_config config;
    nn_input inData;
    nn_output *outdata = NULL;
    int ret;
    int i;
    nn_query *query;
    input_path = (const char *)argv[2];
    config.modelType = TENSORFLOW;
    /***** load adla from memory *****/
    fp = fopen(argv[1],"rb");
    fseek(fp,0,SEEK_END);
    size = (int)ftell(fp);
    rewind(fp);
    config.pdata = (char *)calloc(1,size);
    fread((void*)config.pdata,1,size,fp);
    config.nbgType = NN_ADLA_MEMORY;
    config.size = size;
    fclose(fp);
    /***** load adla from file *****/
    config.path= (const char *)argv[1];
    config.nbgType = NN_ADLA_FILE;

    context = aml_module_create(&config);
    rawdata = read_file(input_path, &input_size);
    inData.input_index = 0; //this value is index of input,begin from 0
    inData.size = 224*224*3;
    inData.input = rawdata;
    inData.input_type = BINARY_RAW_DATA;
    ret = aml_module_input_set(context,&inData);
    if(rawdata != NULL)
    {
```

```

        free(rawdata);
        rawdata = NULL;
    }
    outdata = (nn_output *)aml_module_output_get_simple(context);
    if(outdata->out[0].param->data_format == NN_BUFFER_FORMAT_FP32)
    {
        process_top5((float*)outdata->out[0].buf,outdata->out[0].size/sizeof(float),NULL);}
        ret = aml_module_destroy(context);
        return ret;
    }
    Outdata contains complete output information, including output dim, output format, output content and
    so on. The output content is float32 format by default, which can be implemented after relevant network
    post-processing. Such as mobilenet:
    process_top5((float*)pout->out[0].buf,pout->out[0].size/sizeof(float));
    void process_top5(float *buf,unsigned int num,img_classify_out_t* clsout)
    {
        int j = 0;
        unsigned int MaxClass[5]={0};
        float fMaxProb[5]={0.0};

        float *pfMaxProb = fMaxProb;
        unsigned int *pMaxClass = MaxClass,i = 0;

        for (j = 0; j < 5; j++)
        {
            for (i=0; i<num; i++)
            {
                if ((i == *(pMaxClass+0)) || (i == *(pMaxClass+1)) || (i == *(pMaxClass+2)) ||
                    (i == *(pMaxClass+3)) || (i == *(pMaxClass+4)))
                {
                    continue;
                }
                if (buf[i] > *(pfMaxProb+j))
                {
                    *(pfMaxProb+j) = buf[i];
                    *(pMaxClass+j) = i;
                }
            }
        }
        for (i=0; i<5; i++)
        {
            if (clsout == NULL)
            {
                printf("%3d: %8.6f\n", MaxClass[i], fMaxProb[i]);
            }
            else
            {
                clsout->score[i] = fMaxProb[i];
                clsout->topClass[i] = MaxClass[i];
            }
        }
    }
}

```

3.5 Model performance debug interface

```

aml_util_enableProfile(context, &profile_data);
ret = run_network(netType, context, jpath);
aml_util_getProfileInfo(context, &profile_data);

printf("\nProfile data:\n");
fprintf(stdout, "\nProfiling data:\n");
fprintf(stdout, "model_name: %s\n", config.path);

fprintf(stdout, " -----From system-----\n");
fprintf(stdout, " core_freq_cur          : %llu Hz\n", profile_data.profiling_data.ext.core_freq_cur);

```

```

    fprintf(stdout, "    axi_freq_cur      : %llu Hz\n", profile_data.profiling_data.ext.axi_freq_cur);
    fprintf(stdout, "    mem_pool_size      : 0x%08lX Bytes\n",
profile_data.profiling_data.ext.mem_pool_size);
    fprintf(stdout, "    mem_pool_used      : 0x%08lX Bytes\n",
profile_data.profiling_data.ext.mem_pool_used);
    fprintf(stdout, "    mem_allocated_base : 0x%08lX
Bytes\n",profile_data.profiling_data.ext.mem_allocated_base);
    fprintf(stdout, "    mem_allocated_umd   : 0x%08lX Bytes\n",
profile_data.profiling_data.ext.mem_allocated_umd);
    int elapsed_hw_sw = (int)profile_data.profiling_data.ext.us_elapsed_in_hw_op +
(int)profile_data.profiling_data.ext.us_elapsed_in_sw_op;

    fprintf(stdout, "-----aml need-----\n");
    fprintf(stdout, "    : %7.2f ms, inference_time\n", (float)profile_data.profiling_data.inference_time_us
/ 1000);
    fprintf(stdout, "    : %7.2f ms, elapsed_in_hw_op  \n",
(float)profile_data.profiling_data.ext.us_elapsed_in_hw_op / 1000);
    fprintf(stdout, "    : %7.2f ms, elapsed_in_sw_op  \n",
(float)profile_data.profiling_data.ext.us_elapsed_in_sw_op / 1000);
    fprintf(stdout, "    : %7.2f ms, elapsed (hw_op+sw_op)  \n", (float)elapsed_hw_sw/1000);
    fprintf(stdout, "    : %7.2f ms, invoke_time          \n", (float)usVal_inference / 1000);
    fprintf(stdout, "    : %7.2f  FPS(pm)                  \n",(double)1000000 /
(double)profile_data.profiling_data.inference_time_us);
    fprintf(stdout, "    : %7.2f  FPS(hw+sw)               \n", (double)1000000 / (double)(elapsed_hw_sw));
    fprintf(stdout, "    : %7.2f  FPS(invoke) \n", (double)1000000 / (double)(usVal_inference));
    fprintf(stdout, "    : %7.2f Mbytes,  dram_read_size
\n", (float)profile_data.profiling_data.dram_read_bytes / (1024 * 1024));
    fprintf(stdout, "    : %7.2f Mbytes,  dram_write_size
\n", (float)profile_data.profiling_data.dram_write_bytes / (1024 * 1024));
    fprintf(stdout, "    : %7.2f Mbytes,  BW(Ddr_data)
\n", (double)(profile_data.profiling_data.dram_read_bytes +
profile_data.profiling_data.dram_write_bytes) / (1024 * 1024));
    fprintf(stdout, "    : %7.2f Mbytes,  memory_usage_bytes
\n", (float)profile_data.profiling_data.memory_usage_bytes / (1024 * 1024));

    aml_util_disableProfile(context, &profile_data);

```